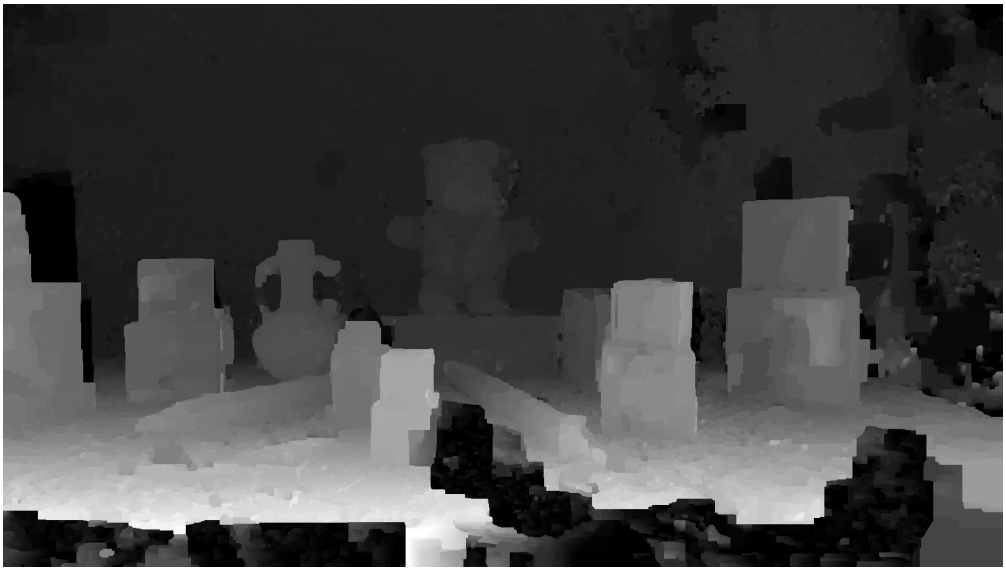
Université Libre de Bruxelles

INFO—H503

GPU computing



CUDA Planesweep Project

Antoine Berthion — 566199

May 24, 2026

Supervised by:
Bonatto Daniele and Soetens Eline

Table of contents

1 Introduction

Depth estimation is a fundamental problem in computer vision and lies at the core of many important applications, ranging from *environment perception* in robotics to various *3D modeling* use cases. It consists of estimating the distance between objects and a camera in a given scene by exploiting images captured from **different viewpoints**.

In this report, we present the *Plane Sweep* algorithm, a method used to compute a **depth map**, which is a grayscale representation of the original scene where darker pixels correspond to objects that are farther away from the camera. The algorithm works by projecting the scene onto a series of **hypothetical planes** and evaluating the consistency of color matching between different camera views.

We will first discuss the theoretical foundations of the algorithm, then present possible CPU and GPU implementations, along with several optimization techniques and benchmarking results.

2 Depth Estimation

In this section, we present the problem of depth estimation from multiple camera viewpoints. We first introduce the *Plane Sweep* algorithm, which is used to construct a cost volume representing the likelihood of different depth hypotheses for each pixel. We then describe two depth extraction methods used to generate the final depth map from this cost volume, which are used in our project.

2.1 Plane Sweep Algorithm

The Plane Sweep algorithm is a method used to estimate the depth of a scene from several calibrated camera images. The main idea is to evaluate a set of hypothetical depth planes and determine, for each pixel, which depth hypothesis provides the best correspondence between different camera views.

The algorithm starts by selecting a **reference camera**. For every pixel in this reference image, a series of candidate depth planes is tested between a near plane Z_{Near} and a far plane Z_{Far}. The sampled depth associated with a plane index z_i is computed using a projective distribution in order to obtain a denser sampling for nearby objects:

$$z = \frac{Z_{Near} \cdot Z_{Far}}{Z_{Near} + \left(\frac{z_i}{N_{Planes}}\right)(Z_{Far} - Z_{Near})}$$

For each candidate depth, the corresponding 3D point is reconstructed in the reference camera coordinate system using the inverse intrinsic matrix $\mathbf{K}^{-1}$. The point is then transformed into world coordinates using the inverse rotation and translation matrices of the reference camera. Finally, this 3D point is projected into the coordinate system of another camera using its intrinsic and extrinsic parameters. See figure 1.

This reprojection process allows us to determine where a pixel from the reference image would appear in another image if the current depth hypothesis were correct. If the depth assumption is accurate, the projected pixels should correspond to the same physical point in the scene and therefore exhibit similar intensity values.

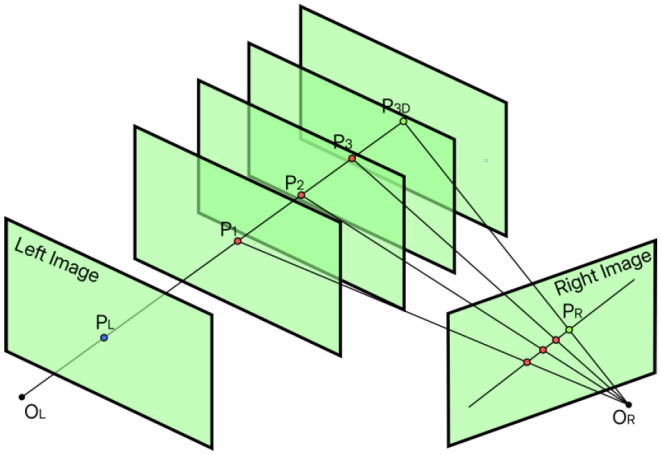


Figure 1: Planesweep algorithm

To evaluate this similarity, a matching cost is computed between the reference image and the projected image. In our implementation, we use the `Y` channel of the YUV color space and compute the average absolute difference over a square window centered around the pixel:

$$C(p, z_i) = \frac{1}{|W|} \sum_{q \in W} |I_{ref}(q) - I_{proj}(q)|$$

where W represents the matching window around pixel p.

For each depth plane, the resulting matching costs are stored inside a three-dimensional structure that will be referred to as the **cost cube** in our project. Each layer of this volume corresponds to the matching cost associated with one candidate depth plane.

Since multiple auxiliary cameras are used, the minimum matching cost across all views is retained for every pixel and every depth hypothesis. This approach improves robustness against occlusions and inaccurate matches.

2.2 Depth Extraction

Once the **cost cube** has been computed, a depth value must be assigned to each pixel to generate a **Disparity Space Image** (our grayscale image). Two different approaches are implemented in this project.

2.2.1 Argmin approach

The simplest approach consists of selecting, for every pixel, the depth plane that **minimizes** the matching cost. For each pixel (x, y), the selected depth is defined as:

$$D(x, y) = \arg\min_{z_i} C(x, y, z_i)$$

This approach is computationally efficient and easy to implement. However, because each pixel is processed independently, the resulting depth maps are often noisy or granular and may contain discontinuities or unstable regions in low-texture areas.

2.2.2 Graph-Cut Optimization

To obtain smoother and more coherent depth maps, a second approach based on graph-cut optimization is also implemented.

In this method, the depth estimation problem is formulated as an energy minimization problem composed of two terms:

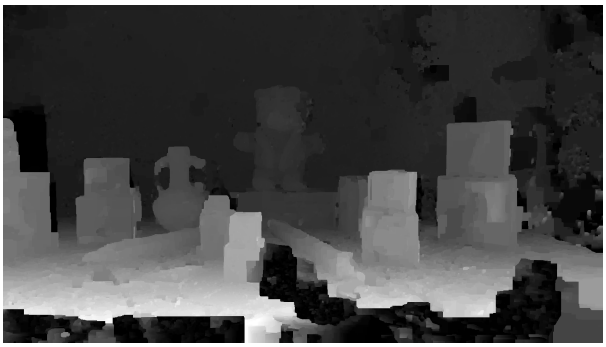
- a **data term**, corresponding to the matching cost extracted from the cost volume,
- a **smoothness term**, encouraging neighboring pixels to have similar depth values to smoothen the image.

A graph is constructed where each pixel corresponds to a node connected to a source and a sink. Additional edges are added between neighboring pixels in order to penalize abrupt depth variations. The optimal labeling is then obtained by solving a *maximum-flow/minimum-cut* problem.

2.3 Comparison of Depth Extraction Methods

In this subsection, we compare the two depth extraction methods introduced in Sections 2.2.1 and 2.2.2. The results are obtained using a **plane sweep with 256 depth planes**.

(a) Argmin approach



(b) Graph-cut optimization

Figure 2: Comparison of depth extraction methods

As discussed previously, Figure 2a produces a noticeably more granular and noisy depth map. In contrast, the graph-cut result shown in Figure 2b is significantly smoother and more spatially consistent. However, this improvement comes at a much higher computational cost, since the graph-cut method requires solving a hard optimization problem.

3 Parallelization

In this section, we present the GPU implementation of the Plane Sweep algorithm, focusing on how the computation is parallelized across depth planes, pixels, and source cameras. We also briefly discuss the parallelization strategy used for the argmin-based depth extraction method introduced in Section 2.2.1.

3.1 Parallelization Strategy for Plane Sweep

The Plane Sweep algorithm is particularly well-suited for parallel execution due to the independence of its core computations. Each pixel in the reference image can be processed independently, and each depth

hypothesis can be evaluated separately. This naturally leads to a three-dimensional parallel structure over image width, image height, and depth planes, using three-dimensional grids.

In the GPU implementation, each thread is assigned a specific combination of pixel coordinates (x, y) and a depth plane index z_i. This allows the algorithm to compute the matching cost for a single hypothesis in parallel with all other hypotheses across the image.

For each thread, the algorithm performs the same sequence of operations as in the CPU version: the reference pixel is first unprojected into 3D space using the camera intrinsics and the current depth hypothesis. The resulting 3D point is then transformed into the world coordinate system and reprojected into each source camera. Finally, a similarity measure is computed between the reference image and the projected source image over a local window.

Since multiple source cameras are available, each thread aggregates the matching cost across all views and retains the minimum cost. This step ensures robustness against occlusions and viewpoint-dependent errors.

The full cost volume is therefore computed in a massively parallel manner, where each thread contributes exactly one value to the 3D cost structure. This removes the need for **nested CPU loops** over pixels, depth planes, and camera views, and significantly reduces computation time.

3.2 Parallelization of Depth Extraction (Argmin)

Once the cost volume has been computed, the depth extraction step consists of selecting, for each pixel, the depth plane that minimizes the matching cost.

This operation is also highly parallelizable, since each pixel can independently search for its best depth without requiring information from neighboring pixels. In the GPU implementation, each thread is assigned a single pixel and iterates over all depth planes to find the minimum cost.

Although this step is less computationally expensive than cost volume construction, its parallel execution ensures that no sequential bottleneck remains in the pipeline, allowing the full depth estimation process to be executed efficiently on the GPU.

4 Optimizations

This section presents the main optimizations applied to the naïve GPU implementation of the Plane Sweep algorithm. The focus is not on the kernels themselves, but rather on the memory and data-layout optimizations introduced to improve memory access efficiency and overall GPU performance.

4.1 Data types

One of the first optimizations concerns the choice of data structures and data types. In the original CPU implementation, most matrices are stored using `cv::Mat` or `std::vector`, and more complex structures such as `cam` contain additional abstraction and memory overhead.

For the GPU implementation, these structures were replaced whenever possible by simpler plain structures and contiguous arrays. This reduces memory overhead and improves memory locality, which is important for efficient GPU memory accesses.

Another important optimization is the use of `float` instead of `double`. A `float` uses **32 bits** of memory, while a `double` uses **64 bits**. Besides reducing memory usage and bandwidth, most consumer GPUs are significantly faster when using single-precision arithmetic. This is because they contain many more FP32 units than FP64 units, as illustrated in Figure 3. The tradeoff is a small loss in numerical precision, which is acceptable for our use case.

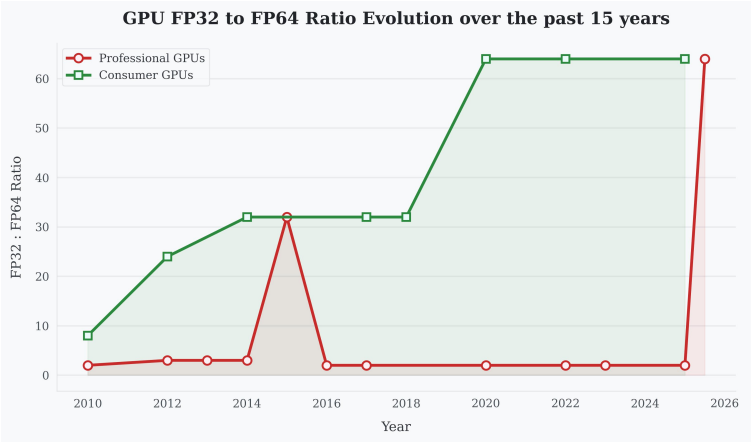


Figure 3: Comparison between FP32 and FP64 throughput on recent GPUs

4.2 LDG Cache

Another important optimization used in our GPU implementation is the use of the LDG (Load Global) cache mechanism. In CUDA, the `__ldg()` intrinsic allows read-only data to be fetched through a dedicated read-only cache instead of the standard global memory path.

This is particularly useful when data is only read and never modified, which is the case for our images pixels. By marking these memory accesses as read-only, the GPU can take advantage of the LDG cache to reduce global memory bandwidth pressure and improve effective memory latency. See Figure 4.

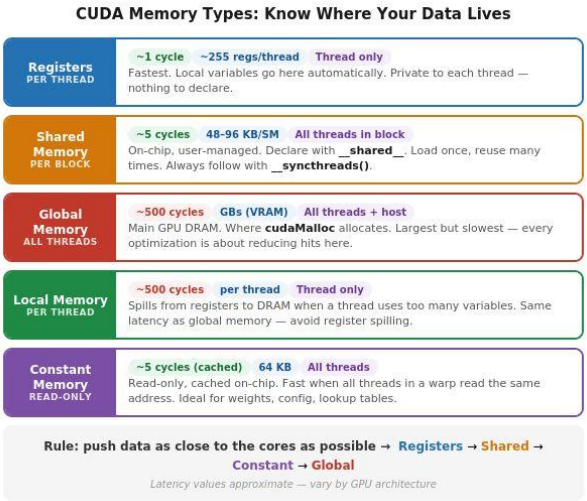


Figure 4: CUDA Memory Types

In practice, this helps improve performance when multiple threads access the same data, since cached values can be reused without repeatedly accessing global memory. This is especially beneficial in memory-bound kernels, where performance is limited more by memory throughput than by arithmetic operations.

4.3 Shared Memory

Another key optimization is the use of *shared memory* to reduce redundant global memory accesses during the matching cost computation. Shared memory is a small, low-latency memory space shared by all threads within a CUDA block, making it significantly faster than global memory when data reuse is high.

In our implementation, the reference image is loaded into a shared memory tile at the beginning of each block execution. Each block allocates a padded region that includes the pixels required for the matching window (i.e., a halo region of size proportional to the window radius). This ensures that all threads within the block can access the required neighborhood pixels without repeatedly fetching them from global memory.

The loading is performed cooperatively: threads in the block collaboratively copy the relevant portion of the reference image into shared memory. Once the tile is fully loaded, a synchronization barrier (`__syncthreads()`) ensures that all threads can safely access the shared data.

This design is particularly effective because each pixel participates in a 5×5 (or generally window-sized) matching operation, meaning the same reference pixels are reused multiple times across different cost computations. By storing them in shared memory, we significantly reduce redundant global memory traffic and improve memory bandwidth utilization.

The shared memory tile is allocated dynamically based on the block size and window radius:

$$\text{tile width} = \text{blockDim.x} + 2 \cdot r, \quad \text{tile height} = \text{blockDim.y} + 2 \cdot r$$

where $r = \frac{\text{window}}{2}$.

Overall, this optimization improves performance by increasing data locality and reducing expensive global memory accesses, which are often the main bottleneck in memory-bound CUDA kernels.

4.4 Work per Block

To improve GPU utilization and reduce redundant memory operations, each CUDA block is responsible for computing multiple depth planes instead of a single one. In our implementation, we introduce a parameter `ZPlanesPerBlock` set to 2, meaning that each block processes two depth layers along the depth (z) dimension.

This design allows the same shared memory reference tile to be reused across multiple depth hypotheses. Since the reference image tile is independent of the depth value, loading it once per block and reusing it for several depth computations reduces memory traffic and improves overall efficiency.

As a result, threads within a block not only cooperate spatially (over image pixels) but also temporally across depth planes. This improves data reuse and reduces the overhead of repeatedly fetching the same reference data from global or global-cached memory.

Overall, this optimization increases arithmetic intensity per memory load and helps better amortize the cost of shared memory initialization across multiple depth evaluations.

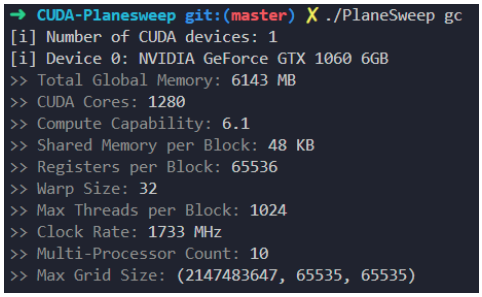
5 Experimentation

In this section, we empirically evaluate the impact of our optimizations, showing improved memory bandwidth usage and a significant speedup, ideally by at least one to two orders of magnitude. We briefly present the specifications of our CUDA-capable device as well as our CPU and discuss the observed performance improvements.

5.1 Device Specifications

The CUDA-capable device used in this experiment is an **NVIDIA GTX 1060**, running under **WSL** (Windows Subsystem for Linux) on an `Ubuntu 24.04.2 LTS environment`. The CPU is a **AMD Ryzen 5 9600X**, also running on a WSL. The full hardware specifications are shown in Figure 5.

(a) AMD Ryzen 5 9600X specifications



(b) GTX 1060 specifications

Figure 5: Hardware specifications

5.2 Methodology

In order to obtain meaningful results, we define the following experimental methodology. On the hardware specified in Section 5.1, we evaluate:

— Naïve CPU implementation,

— Naïve GPU implementation,

— Optimized GPU implementation.

For each implementation, **kernel execution time** is measured. To avoid initialization overhead affecting the results, the first kernel launch is discarded. Each experiment is then repeated **10 times**, and the average execution time is reported as the final benchmark value. Argmin depth extraction will be used for all 3 of these.

We will finally verify that all three implementations produce very similar results in order to ensure consistency.

5.3 Results

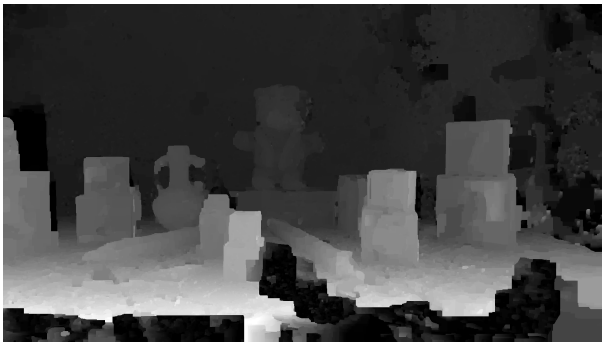
The results of our experiments **over 10 iterations** can be found below in Table 1.

Implementation	PlaneSweep time	Argmin time	Total time	Speedup
Naïve CPU	442 s	1681 ms	455 s	1×
Naïve GPU	4361 ms	1683 ms	6 s	75.8×
Optimized GPU	722 ms	12 ms	1.909 s	238.4×

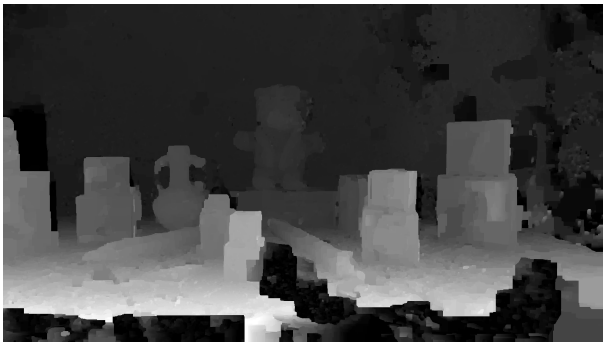
Table 1: Performance comparison of implementations of Plane Sweep

Overall, we observe an approximately 240× **speedup** compared to the CPU implementation. We also

present the depth maps obtained using the CPU and optimized GPU implementations to verify that the results remain consistent. See Figure 6.



(a) Naïve CPU Plane Sweep depth map



(b) Optimized GPU Plane Sweep depth map

Figure 6: Comparison of depth maps produced by CPU and GPU implementations

As we can see, the results remain coherent throughout the optimizations that were applied.

6 Possible Improvements

Several improvements could further enhance the performance of our implementation. First, more aggressive use of shared memory and better tiling strategies could reduce redundant global memory accesses even further. We could also improve the efficiency of our graph-cut optimization by identifying and parallelizing additional parts of the routine, which currently remain sequential and limit overall performance.

On the algorithmic side, reducing the number of depth planes adaptively instead of using a fixed sampling could significantly lower computation cost without strongly affecting accuracy. Finally, more advanced GPU techniques such as warp-level primitives or mixed-precision strategies could also provide additional speedups in future versions. However, it goes outide of the scope of our project.

7 Use of LLMs

In this brief section, we discuss the use of large language models (LLMs) in the context of this project. No LLM was used for the implementation or for understanding the project itself. However, some documentation in the project and this report were reviewed for syntax and grammar by DeepL as well as a GPT model. It should be noted that none of the information contained in this report was generated by anyone other than the author.

8 Conclusion

In this project, we implemented a Plane Sweep stereo reconstruction pipeline and progressively optimized it for GPU execution. Starting from a naïve CPU version, we developed a CUDA-based implementation and introduced several memory-oriented optimizations, including data type simplification, LDG caching, shared memory tiling, and improved work distribution across blocks.

These optimizations significantly reduced memory overhead and improved parallel efficiency, leading to a substantial speedup by orders of magnitude ($240\times$) compared to the baseline CPU implementation while preserving visually consistent depth maps.

References

[1] Richard Szeliski. *Image Formation.* Computer Vision Algorithms and Application, `Springer`, chap. 2, 2011.

[2] Richard Szeliski. *Stereo Correspondance.* Computer Vision Algorithms and Application, `Springer`, chap. 11, 2011.